



Data Cleaning in R: Missing Values and Outlier Detection Explained

Data Cleaning: Facing Reality

Real-world data is inevitably "dirty" — the result of instrumental failures, human or computational errors, and transmission problems. Identifying the different types of imperfections is the first step toward choosing the most appropriate cleaning techniques.



Incomplete Data

Absence of values in essential attributes, entire attributes not recorded, or only aggregated data available.

- Example: Occupation = "" (blank field)
- Entire attributes may be completely missing



Noisy Data

Errors, noise, or outliers that distort analysis and compromise the reliability of results.

- Example: Salary = "-10" (clearly invalid value)
- Records with physically impossible or implausible values



Inconsistent Data

Discrepancies between related fields, conflicting duplicate records, or changes in coding schemes over time.

- Example: Age = "42", Date of Birth = "03/07/2010"
- Migration from numeric to alphabetic classification (1,2,3 → A,B,C)



Intentionally Incorrect Data

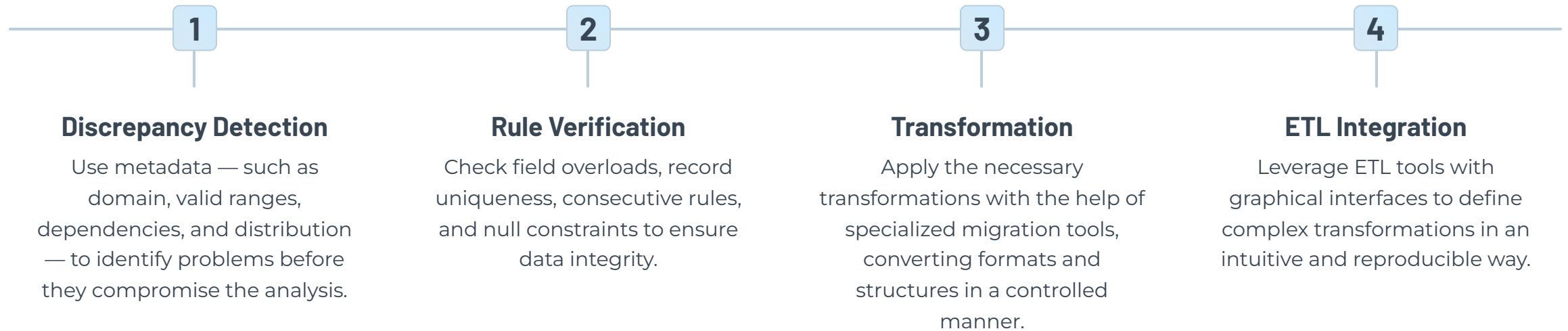
Information deliberately hidden or replaced with generic values to mask sensitive data.

- Example: January 1st as the default birth date for everyone
- Substitute values used to conceal actual information

 **Reference:** Chu, Ilyas, Krishnan & Wang (2016). Data cleaning: Overview and emerging challenges. ACM SIGMOD.

Data Cleaning as a Process

Data cleaning goes beyond a one-time task — it is a systematic and iterative process that requires careful planning and appropriate tools. Its steps are interrelated and, together, ensure the quality and reliability of data over time.



Tools and Technologies

Data Migration Tools: Allow transformations to be specified declaratively, simplifying conversion between different data formats and structures.

ETL Tools (Extraction/Transformation/Loading): Offer graphical interfaces that make defining complex transformations accessible to users with different technical backgrounds, significantly reducing operational complexity.

📖 **References:** Chu et al. (2016). Data cleaning: Overview and emerging challenges. ACM SIGMOD. • Wang & Wang (2020). Time Series Data Cleaning: A Survey. IEEE Access, 8.

Overview – Cleaning

`na_removal`

Removes all rows containing missing values (NA). Follows the DAL pattern — instantiate the object and apply `transform()` — making it the `daltoolbox` equivalent of base R's `na.omit`. Works well when the proportion of NAs in the dataset is small.

`outliers_boxplot`

Identifies and discards outliers based on the IQR criterion, treating values beyond $1.5 \times$ the interquartile range as atypical. It is a robust strategy, especially suitable for asymmetric distributions.

`outliers_gaussian`

Detects outliers by assuming a normal distribution, flagging as atypical the values that deviate more than k standard deviations from the mean. It is the natural choice when the data approximately follows a Gaussian distribution.

- ❏ **General principle:** Cleaning should be applied exclusively to training data. Never use information from the test set to decide which observations to remove — this would constitute data leakage.

Missing Values – na_removal

Missing values (NA) are common in real-world datasets, arising from various sources like sensor failures, data entry errors, or incomplete surveys. They can significantly impact data analysis and model performance.

The `na_removal` function from **daltoolbox** employs a strategy known as **listwise deletion**, where any row containing at least one missing value is entirely removed from the dataset. This is the simplest approach to handling NAs.

This method is appropriate when the proportion of missing values is very small and the missingness occurs **completely at random (MCAR)**. If missingness is not random, removing rows can introduce bias into the analysis and reduce statistical power, leading to skewed conclusions.

`na_removal()` follows the standard DAL pattern: first create the object with `na_removal()`, then apply it to the data using `transform(obj, data)`. This interface is consistent with other cleaning functions such as `outliers_boxplot` and `outliers_gaussian`, making it easy to swap strategies within the same pipeline.

MCAR (Missing Completely at Random)

The probability of missing data for a variable is unrelated to the value of any variable, observed or unobserved.

MAR (Missing at Random)

The probability of missing data for a variable is related to observed variables, but not to the missing variable itself.

MNAR (Missing Not at Random)

The probability of missing data depends on the value of the missing variable itself, even after controlling for other observed variables.

📖 **Reference:** Little, R.J.A. & Rubin, D.B. (2002). Statistical Analysis with Missing Data (2nd ed.). Wiley.

NA Removal – na_removal

```
library(daltoolbox)

iris <- datasets::iris
nrow(iris) # [1] 150

# Introducing an NA artificially
iris.na <- iris
iris.na$Sepal.Length[2] <- NA
nrow(iris.na) # [1] 150

# Creating the na_removal object
obj <- na_removal()
```

```
# Applying the transformation
iris.clean <- transform(obj, iris.na)
nrow(iris.clean) # [1] 149

head(iris.clean)
# Sepal.Length Sepal.Width Petal.Length ...
# 1      5.1      3.5      1.4
# 3      4.7      3.2      1.3
# 4      4.6      3.1      1.5
# (row 2 removed)
```

Step by step

- An NA is inserted into `Sepal.Length[2]` to simulate a missing value in the dataset
- `na_removal()` creates a DAL transform object following the same interface as `outliers_boxplot` and `outliers_gaussian`
- `transform(obj, iris.na)` applies listwise deletion — any row with at least one NA is removed
- The dataset shrinks from 150 to 149 instances — row 2 no longer exists
- Caution: if missingness is not random (MAR/MNAR), removal may introduce bias into the analysis

📌 **Key concept:** `na_removal` is ideal for a quick initial cleanup following the DAL pattern. When missing data is not random (MCAR, MAR, or MNAR patterns), imputation techniques tend to be more appropriate. (Little & Rubin, 2002)

Outlier Detection – outliers_boxplot

Outliers are extreme values that deviate significantly from other observations in a dataset. They can skew statistical analyses, invalidate assumptions of some models, and lead to erroneous conclusions if not handled appropriately, impacting the reliability of insights.

The `outliers_boxplot` method, often referred to as Tukey's Fences, is a robust technique for identifying outliers based on the interquartile range (IQR). This non-parametric approach leverages the first quartile ($Q1$), the third quartile ($Q3$), and the IQR (calculated as $Q3 - Q1$) to define data boundaries.

Specifically, values falling below $Q1 - 1.5 \cdot IQR$ or above $Q3 + 1.5 \cdot IQR$ are flagged as outliers. These dynamic boundaries, known as the lower and upper fences, adapt to the data's spread without assuming a specific distribution.

Compute Q1 & Q3

Identify the first (25th percentile) and third (75th percentile) quartiles of the data.

Calculate IQR

Determine the Interquartile Range as the difference between the third and first quartiles ($Q3 - Q1$).

Define Fences

Set the lower fence at $Q1 - 1.5 \cdot IQR$ and the upper fence at $Q3 + 1.5 \cdot IQR$.

Flag Outliers

Mark any data points that fall outside these calculated fences as potential outliers.

Remove Flagged Rows

Eliminate or treat the identified outliers from the dataset based on the analysis strategy.

A key advantage of this method is its non-parametric nature, meaning it does not assume any underlying data distribution. This makes it particularly robust for asymmetric distributions or data with heavy tails where other methods might fail. However, it can be aggressive for normally distributed data, potentially flagging approximately 0.7% of observations as outliers even when they are part of the true distribution.

 **Reference:** Tukey, J.W. (1977). Exploratory Data Analysis. Addison-Wesley.

Outliers via Boxplot – outliers_boxplot

```
library(daltoolbox)

iris <- datasets::iris

# Creating and fitting the object
out_obj <- outliers_boxplot()
out_obj <- fit(out_obj, iris)
iris.clean <- transform(out_obj, iris)

nrow(iris.clean) # [1] 146
```

```
# Identifying the outliers
idx <- attr(iris.clean, "idx")
print(table(idx))
# FALSE TRUE
# 146 4

iris.outliers <- iris[idx, ]
head(iris.outliers)
# row 16: Sepal.Width = 4.4 (setosa)
# row 33: Sepal.Width = 4.1 (setosa)
# row 34: Sepal.Width = 4.2 (setosa)
# row 61: Sepal.Width = 2.0 (versicolor)
```

Step by step

- Instantiate `outliers_boxplot()` and fit it with `fit()`, which internally computes Q1, Q3, and the IQR for each variable
- Then, `transform()` returns the already-filtered dataset — in this case, 146 of the original 150 rows
- The `idx` attribute flags which rows were removed: a total of 4 observations identified as outliers
- The criterion used is Tukey's rule: any value outside the interval $[Q1 - 1.5 \cdot IQR, Q3 + 1.5 \cdot IQR]$ is marked as atypical

❏ **Key concept:** The boxplot rule (Tukey, 1977) is non-parametric and robust, as it does not assume a normal distribution — making it especially suitable for skewed data or data with heavy tails.

Outlier Detection – outliers_gaussian

The `outliers_gaussian` method leverages the assumption of a normal (Gaussian) distribution to identify extreme values. It applies the 3σ (three-sigma) rule, classifying any data point falling outside the interval $[\bar{x} - 3\sigma, \bar{x} + 3\sigma]$ as an outlier. Here, $\bar{x}$ represents the mean and σ is the standard deviation of the data.

This method is notably more conservative than approaches like the boxplot method; under a perfectly normal distribution, only approximately 0.27% of observations are expected to fall beyond $\pm 3\sigma$. It is best preferred when the data is known or has been verified to follow an approximately normal distribution. However, its main limitation is its sensitivity to this assumption — if the data is skewed or heavy-tailed, the method may either fail to detect true outliers or incorrectly flag normal observations as outliers, leading to false positives.

outliers_gaussian

- **When to use:** When data is approximately normally distributed.
- **Key assumption:** Data follows a Gaussian distribution.
- **Sensitivity:** Conservative; flags ~0.27% of values as outliers in normal data. Sensitive to non-normality.

outliers_boxplot

- **When to use:** Robust for non-normal, skewed, or heavy-tailed data.
- **Key assumption:** Non-parametric; no specific distribution assumed.
- **Sensitivity:** Less conservative; flags ~0.7% of values as outliers in normal data. Robust to non-normality.

📖 **Reference:** Pukelsheim, F. (1994). The Three Sigma Rule. The American Statistician, 48(2), 88–91.

Gaussian Outliers – outliers_gaussian

```
library(daltoolbox)
iris <- datasets::iris

# Creating and fitting the object
out_obj <- outliers_gaussian()
out_obj <- fit(out_obj, iris)
iris.clean <- transform(out_obj, iris)

nrow(iris.clean) # [1] 149
```

```
# Identifying the outlier
idx <- attr(iris.clean, "idx")
print(table(idx))
# FALSE TRUE
# 149    1

iris.outliers <- iris[idx, ]
head(iris.outliers)
# row 16: Sepal.Width = 4.4 (setosa)
# Value beyond mean + 3*sd
```

Step by step

- `fit()` estimates the mean and standard deviation of each numeric attribute in the dataset
- By the 3σ rule, any value outside the interval $[\bar{x} - 3\sigma, \bar{x} + 3\sigma]$ is classified as an outlier
- Only 1 outlier is detected — a considerably more conservative criterion than the 4 identified by the boxplot
- The method assumes that the data follow an approximately normal distribution

❏ **Key concept:** The 3σ rule (Pukelsheim, 1994) is more conservative and therefore less sensitive to extreme values than the boxplot. Prefer it when the data have a normal distribution; for skewed data, the boxplot method is more appropriate.



Acknowledgements



Data Analytics Lab

dal.global@grupo.cefet-rj.br | <https://eic.cefet-rj.br/dal>

